RA26-Challenge 6: Material de aprendizaje

DMA, scroll, mapas, window, niveles, IA, coordenadas y animación

Cómo usar esta guía

Este documento **explica** todo lo que necesitas saber para el Challenge 6. El Challenge de entrenamiento solo contiene los ejercicios: aquí está la teoría que los hace posibles.

El juego es **CHELDA**, un mini-Zelda visto desde arriba (*quasi-cenital*). Usa la misma arquitectura que ***CHAsteroids***: entidades, componentes, sistemas con rutinas `_one` y `_update`, escenas y reglas del juego. Lo nuevo es lo que hace falta para un mundo más grande que la pantalla, con música y efectos.

Cómo usar esta guía para aprender

1. **Lee un bloque** (unos 15-20 minutos) con el lápiz en la mano. Haz los dibujos y las cuentas tú: entender un número que has calculado vale más que leerlo.
2. **Haz los ejercicios** del documento de entrenamiento: entre 30 y 65 minutos cada uno: entenderlo, hacerlo en papel, pasarlo al ordenador y depurarlo. La tabla está en el entrenamiento.
3. Si un ejercicio se atasca, vuelve a **su bloque**. Tropezar con una idea es parte de aprenderla.

Cada bloque lleva su ejercicio asociado:

Bloque	Tema	Ejercicio
1	DMA: copiar sprites a la OAM	1
2	Scroll y cámara	2
3	El mapa y las colisiones con el mapa	3
4	El HUD con la window	4
5	Los niveles: formato y carga	5
6	IA básica	6 y 7
7	De coordenadas del mundo a coordenadas de la OAM	8 y 9
8	Animación	10

Arquitectura ECS actualizada

Antes de los bloques hay explicación de la arquitectura. Es igual que la de ***CHAsteroids***, pero mejorada: entidad de 16 bytes, 2 sprites 8x16, `man_entity_for_each_matching`, `engine/` y `assets/` separados, gestión de mapa y niveles y música.

La arquitectura, en dos páginas

Dónde está cada cosa

```
src/
├── main.asm       Bucle principal
├── engine/        SERVICIOS: pantalla, memoria, DMA, joypad, texto, aleatorios
│   ├── gfx/       Cargar tiles y paletas
│   ├── snd/       Sonido (bloque final)
│   ├── ui/        El HUD
│   └── lib/       Código de terceros (el driver de música)
├── assets/        DATOS: gráficos (gfx), música y sfx (snd), niveles y mapas (levels)
├── man/           GESTORES: entidades, mapa, escenas
├── sys/           SISTEMAS: un fichero cada uno
└── scn/           ESCENAS: menú, juego, game over, victoria (y las reglas del juego)
```

Hay una idea que lo ordena: **la lógica y los datos van separados**. Si quieres cambiar partes, esta es la idea:

- `assets/` → Cambiar datos del juego: otro nivel, otro dibujo, otra canción.
- `engine/` → Cambiar funciones y lógica que permiten hacer juegos.
- `man/`, `sys/`, `scn/` → Cambiar la lógica del juego (rutinas ECS específicas).

La entidad: 16 bytes

En ***CHAsteroids*** eran 8 bytes. Hacen falta más datos (dirección, imagen, tiempos...). Ampliamos a **16 bytes**.

La primera columna es el ***offset*** de cada campo: a cuántos bytes está del principio de la entidad. En español sería el *desplazamiento*, pero *offset* es el término que verás en toda la documentación y en todo el código, así que mejor acostumbrarse a llamarlo así. Por ejemplo, `X` tiene offset 2: está en la dirección de la entidad + 2 (`E_X` = 2).

Offset	Campo	Para qué sirve	Componente
0	`CMP`	Qué componentes tiene (bit flags). `CMP = 0` : espacio libre	
1	`Y`	Posición vertical **en el mundo** (píxeles, 0-255)	`SPRITE`
2	`X`	Posición horizontal en el mundo	`SPRITE`
3	`TILE`	Primer tile de la imagen que muestra ahora	`SPRITE`
4	`ATTR`	Volteos (y paleta) de sus sprites	`SPRITE`
5	`VY`	Velocidad vertical, con signo	`PHYSICS`
6	`VX`	Velocidad horizontal, con signo	`PHYSICS`
7	`KIND`	Qué es (bit flags)	`COLLIDER`
8	`DIR`	Hacia dónde mira: `DIR_DOWN` , `DIR_UP` , `DIR_LEFT` , `DIR_RIGHT`	`ANIMATION`
9	`SHEET`	Primer tile de su sheet de imágenes	`ANIMATION`
10	`TICK`	Frames que lleva caminando (su paso)	`ANIMATION`
11	`LIFE`	Frames que le quedan antes de desaparecer	`SELFDESTROY`
12	`THINK`	Frames que faltan para que decida qué hacer	`AI`
13-15	`RESERVED`	***Libres***. Ajustamos a 16 bytes para que sea potencia de 2.	

Los componentes son bits del byte `CMP` :

```
    bit   7 6 5 4 3 2 1 0
    CMP  |1|0|1|1|0|1|1|1|          CMP_SPRITE      %00000001  CMP_INPUT     %00010000
  ALIVE ─┘ | | | | | | └─  SPRITE   CMP_PHYSICS     %00000010  CMP_ANIMATION %00100000
     AI ─┘ | | | | | └─ PHYSICS     CMP_COLLIDER    %00000100  CMP_AI        %01000000
ANIMATION ─┘ | | └─ COLLIDER       CMP_SELFDESTROY %00001000  CMP_ALIVE     %10000000
      INPUT ─┘ └─ SELFDESTROY
```

Optimización importante: `entity_array` alineado

Las 12 entidades ocupan 192 bytes. Eso permite ***alinearlo*** al tamaño de ***una página*** (256 bytes):

- Con `ALIGN[8]` forzamos que `entity_array` empiece en una dirección `$NN00` (8 bits menores = 0).
- Termina en `$NNC0` . Cualquier entidad está en `$NN__` : su byte alto (`$NN`) nunca cambia.
- Por eso, podemos recorrer las entidades cambiando solo el byte bajo (`$__`), así:

```
;; BC=&entidad[N]; Puntero entidad N del array  ▌ ld h, d    ;; \ ← DE = &entidad
ld   a, c          ;; \                         ▌ ld a, e    ;; |
add  SIZEOF_ENTITY ;; |                         ▌ add E_KIND ;; |
ld   c, a          ;; / ← BC = &entidad[N+1]    ▌ ld l, a    ;; / ← HL = &entidad.KIND
```

- Si `HL=&entidad` , su siguiente campo es `inc l` .

Sistemas: `_one` y `_update`, con filtro

Un sistema son dos rutinas: `sys_xxx_one` procesa **una** entidad (`DE`) y `sys_xxx_update` las procesa **todas**. La novedad es que el **filtro** (qué componentes necesita la entidad) lo hace el *Entity Manager*:

```
sys_xxx_update:
    HL = sys_xxx_one
     C = componentes obligatorios  (REQUIRED_CMPs)
    salta a man_entity_for_each_matching
```

`man_entity_for_each_matching` llama a `sys_xxx_one` **solo** para las entidades vivas que tienen todos los componentes de `C` . Así `sys_xxx_one` se ahorra comprobar si la entidad en `DE` tiene los componentes requeridos: ya lo hace `man_entity_for_each_matching` .

Un frame

```
main (cada frame)
  input_update        lee el joypad
  frame_tick          cuenta un frame
  scene_update        update de escena actual
  sys_render_update   espera VBLANK, copia Shadow OAM a OAM (DMA) y hace scroll
  hud_update          si hace falta, redibuja el HUD (aún en VBLANK)
  sound_update        un tick de la música
```

El juego se actualiza **fuera** de VBLANK (con la pantalla dibujándose) y solo toca la RAM. Todo lo que cambia la VRAM (sprites, scroll, HUD) se hace **en VBLANK**: es el momento más seguro. Como `sys_render_update` espera a VBLANK, también marca el ritmo del juego: unos 60 frames por segundo.

Bloque 1 · DMA: copiar sprites a la OAM

Ejercicio 1 · ~20 minutos de lectura

El problema

Los sprites que ve la consola están en la **OAM**: 40 sprites × 4 bytes = 160 bytes en `$FE00`. Solo se puede escribir en la OAM durante el VBLANK, que dura **unos 1.140 ciclos**. El juego prepara los sprites en una copia en RAM, la **Shadow OAM** (`shadow_oam`), con calma y fuera de VBLANK. En VBLANK hay que copiarla a la OAM.

Copiar 160 bytes con `memcpy8` cuesta 10 ciclos por byte: **1.600 ciclos**. No caben.

La solución: el hardware copia por ti

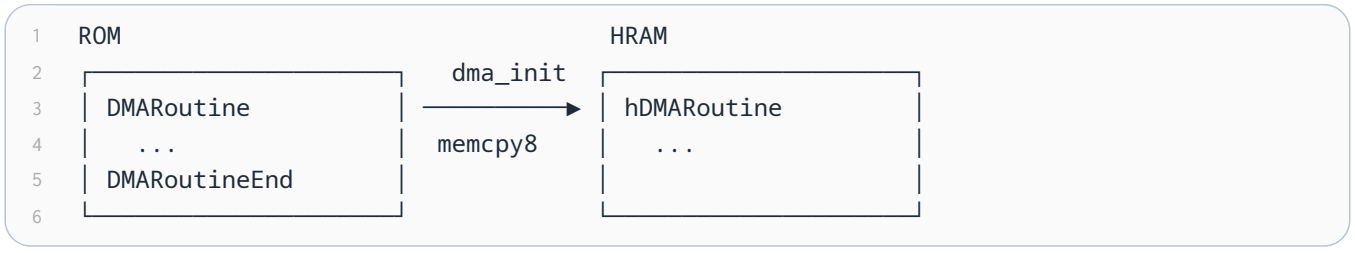
La Game Boy tiene un copiador por hardware, el **DMA** (*Direct Memory Access*). Se usa así:

1. Escribes en el registro `rDMA` (`$FF46`) **la parte alta de una dirección** (la *página*). La Shadow OAM está ***alineada*** a 256 (como `entity_array`), así que su página es `HIGH(shadow_oam)`.
2. El hardware copia 160 bytes desde `$xx00` hasta la OAM. Tarda **160 ciclos**.
3. Mientras copia, **la CPU no puede leer casi nada**: el bus está ocupado. Solo puede leer la **HRAM** (`$FF80-$FFFE`), una zona pequeña de memoria rápida.

Consecuencia: el código tiene que estar en HRAM

Si el código que lanza el DMA está en la ROM, la CPU intentaría leer la siguiente instrucción mientras el bus está ocupado y fallaría. Por eso:

- La rutina que lanza el DMA y espera a que acabe vive **en HRAM** (`hDMARoutine`).
- Su código está guardado en la ROM como un dato (`DMARoutine`, hasta `DMARoutineEnd`).
- `dma_init` la **copia a HRAM una sola vez**, al arrancar. Para saber cuántos bytes copiar, el ensamblador calcula `DMARoutineEnd - DMARoutine`.



Qué hace `DMARoutine`

Tres pasos: lanzar, esperar, volver.

1. **Lanzar**: carga `HIGH(shadow_oam)` en `A` y escribe `A` en `rDMA`. Como `rDMA` está en `$FF46`, puedes usar `ldh [rDMA], a`, que es más corto y rápido que `ld [$FF46], a`.
2. **Esperar** 160 ciclos sin tocar ROM ni WRAM. Un bucle de espera cuenta hacia atrás:

```
        ld   a, 40
.wait   dec  a            ;; 1 ciclo
        jr   nz, .wait    ;; 3 ciclos si salta
```

Cada vuelta cuesta 4 ciclos: `40 × 4 = 160`. Esa es la constante `DMA_WAIT_LOOPS`. Las instrucciones del bucle están en HRAM, así que la CPU puede seguir leyéndolas.
3. **Volver** con `ret`. Quien la llama (`dma_run`) la ejecuta con un `jp` y el `ret` vuelve a su llamador.

Corregir código: cómo leer una rutina buscando errores

El ejercicio 1 te da `DMARoutine` y `dma_init` con errores. No necesitas saber más del DMA que lo que acabas de leer: los errores están en cosas que ya conoces (bucles, retornos, registros y nombres). Una forma de encontrarlos sin dar vueltas:

1. Escribe a un lado **lo que debería hacer cada línea**, con la descripción de arriba.
2. Compara línea a línea con el código. Si hay un bucle, cuenta las vueltas que da con la condición que tiene.
3. Revisa los contratos: `memcpy8` pide el origen en `HL`, el destino en `DE` y el número de bytes en `B`.
4. Ensambla. Un nombre que no existe lo encuentra el ensamblador, pero el resto solo lo encuentras leyendo.

Sitios donde suelen esconderse los errores en este tipo de rutina: la condición del salto del bucle (`jr z` / `jr nz`), cómo termina la rutina (¿vuelve?), qué va en cada registro al llamar a una rutina, y el nombre de las constantes.

Comprueba que lo entiendes

1. ¿Por qué `DMARoutine` no puede llamar a otra rutina de la ROM mientras dura el DMA?
2. Si quisieras copiar la Shadow OAM con `memcpy8` en VBLANK, ¿cuántos ciclos de más te faltarían?

Respuestas al final de esta guía.

Bloque 2 · Scroll y cámara

Ejercicio 2 · ~20 minutos de lectura

El mundo es más grande que la pantalla

La pantalla muestra **160 × 144 píxeles**. El mundo de CHELDA es un mapa de **32 × 32 tiles = 256 × 256 píxeles**, que es justo el tamaño del *background* de la consola. Solo vemos un trozo: un recuadro de 160 × 144 que se mueve por el mundo.

La **cámara** es la posición del mundo donde está la esquina de arriba a la izquierda de lo que se ve:

```
mundo 256 x 256
┌──────────────────────────────┐
│       (X, Y) cámara          │
│         ●──────────┐         │
│         │ pantalla │         │
│         │ 160x144  │         │
│         └──────────┘         │
└──────────────────────────────┘
```

El HUD está en la window, que no se mueve (ver bloque 4).

Mover el background: `rSCX` y `rSCY`

Dos registros dicen qué píxel del background se dibuja arriba a la izquierda de la pantalla:

- `rSCX` (`$FF43`): la X de la cámara.
- `rSCY` (`$FF42`): la Y de la cámara.

Poner `rSCX = cámara X` y `rSCY = cámara Y` **es** el scroll. Como el mundo mide justo lo mismo que el background (256 × 256), cada píxel del mundo está ya en el background y no hay que cargar nada más al moverte.

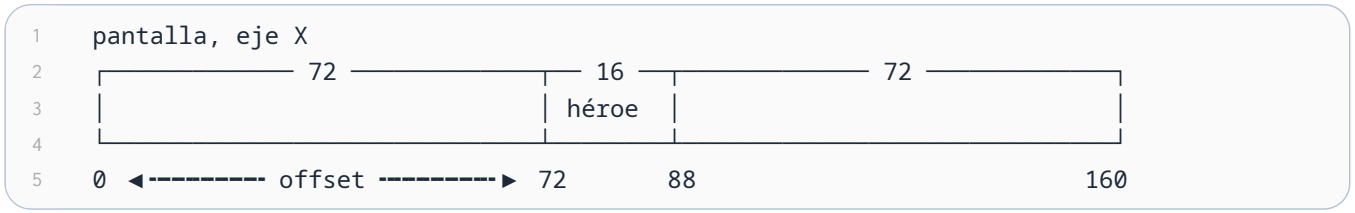
> ⚠ **Solo en VBLANK**
> Si cambias `rSCX` o `rSCY` mientras se dibuja la pantalla, la imagen se parte (verás unas líneas movidas respecto a las demás). Por eso `sys_render_update` hace el scroll justo después de esperar a VBLANK.

La cámara vive en `sys_camera` y es un dato **privado**: `cam_x` y `cam_y` solo los lee y escribe ese sistema. Los demás piden la cámara con `sys_camera_get`, que devuelve `B` = **cámara Y** y `C` = **cámara X**.

Qué cámara queremos

Queremos al héroe en el **centro** de la pantalla. Por eso, calcularemos la posición (X, Y) de la cámara (su esquina de arriba a la izquierda), a partir de la posición (X, Y) del héroe. Como ejemplo, lo vemos en el eje X; en Y es igual.

1. **Dónde tiene que quedar en la pantalla.** La imagen del héroe mide **16** y la pantalla 160. Para que quede centrada, sobra la mitad a cada lado `(160 - 16) / 2 = 72` píxeles. Ese es el *offset*: la distancia entre el borde de la pantalla y la esquina de la imagen del héroe (la misma idea que el offset de un campo: una distancia desde un punto de partida). En Y: `(144 - 16) / 2 = 64`.



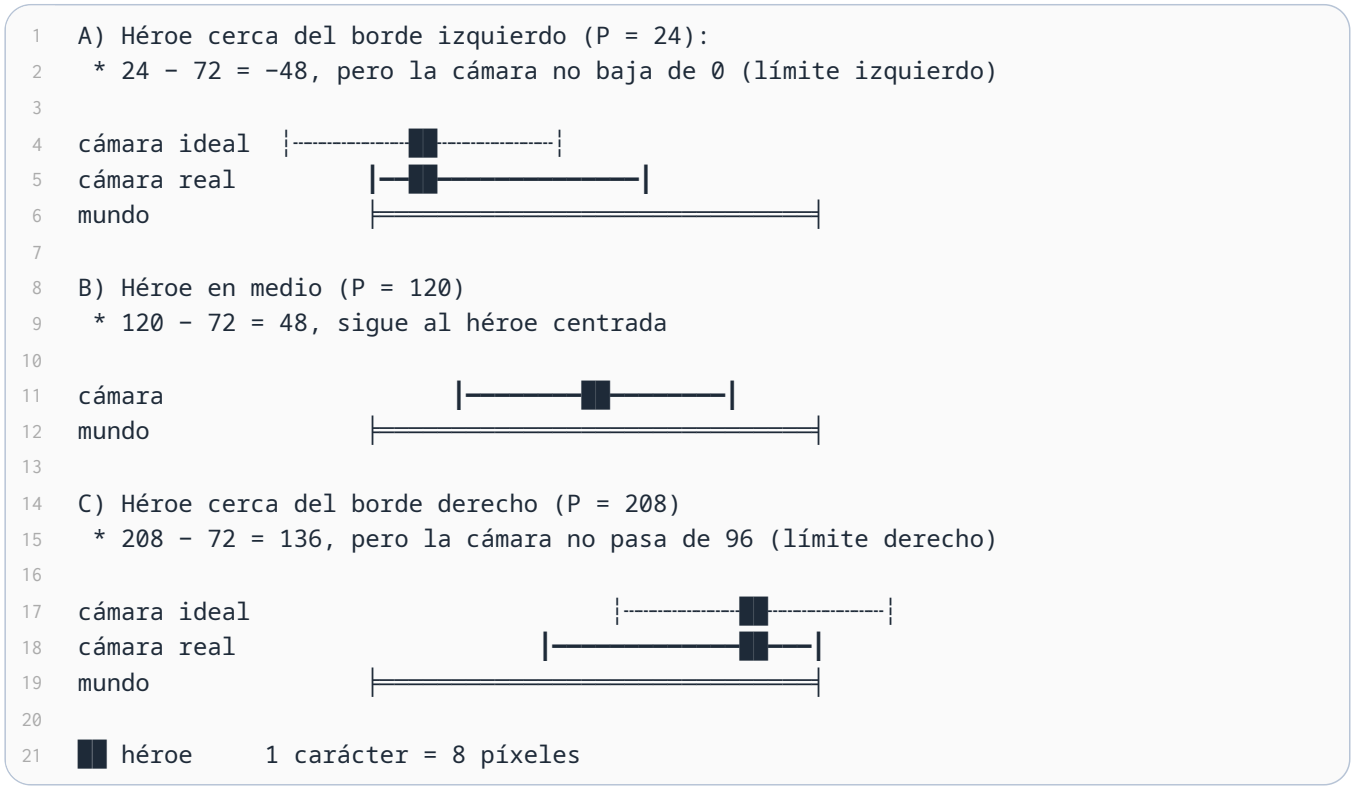
2. **Qué cámara lo consigue.** Lo que se ve en la pantalla es la posición del mundo menos la cámara: `pantalla = mundo - cámara` (lo verás en el bloque 7). Si el héroe está en la posición `P` del mundo y queremos que se dibuje en el offset:

```
offset = P - cámara      →      cámara = P - offset
```

Por ejemplo, con `P = 120` en X la cámara es `120 - 72 = 48`, y el héroe se dibuja en `120 - 48 = 72`: justo en el centro.

Pero hay dos límites, porque no podemos enseñar lo que hay fuera del mundo:

1. La cámara **no baja de 0**.
2. La cámara **no pasa del máximo**: el último sitio donde la pantalla aún cabe en el mundo. En X es `256 - 160 = 96`; en Y, `256 - 144 = 112`.



Restar en 8 bits: el acarreo cuenta cuándo se pasa de 0

En 8 bits, si restas `A - B` y `A < B`, el resultado "da la vuelta" (p. ej. `30 - 72 = 214`) y la CPU activa el **acarreo** (en inglés, *carry*: el flag `C`). Ese acarreo es justo la pregunta "¿me he pasado de 0?". Después de un `sub`, con `jr nc` / `jr c` (o `ret c`) decides según haya o no acarreo.

Para el máximo no hay acarreo que valga: comparas con `cp`. `cp c` hace `A - C` sin guardar el resultado y activa el acarreo si `A < C`.

La rutina de un eje: `camera_axis`

Sirve para X y para Y. En pseudocódigo:

```
camera_axis(A = posición, B = offset, C = máximo):
    A = A - B
    si hubo acarreo (se pasó de 0):  devolver 0
    si A < C:                        devolver A
    devolver C
```

Un detalle: `A` cambia dentro de la rutina, y es lo que devuelve.

La rutina de scroll: `sys_render_scroll`

1. Pide la cámara con `sys_camera_get` (`B` = Y, `C` = X).
2. Escribe `C` en `rSCX` y `B` en `rSCY`. Para escribir un registro hay que pasar por `A`: no existe `ld [rSCX], c`.

Comprueba que lo entiendes

1. El héroe está en `X = 20`. ¿Qué cámara X sale? ¿Y con `X = 200`?
2. ¿Por qué `camera_axis` devuelve 0 si hubo acarreo en vez de devolver el resultado de la resta?

Respuestas al final de esta guía.

Bloque 3 · El mapa y las colisiones con el mapa

Ejercicio 3 · ~20 minutos de lectura

Un nivel es una tabla de tiles

El mapa de un nivel está en la ROM como una tabla de **32 filas de 32 tiles** (1.024 bytes). Cada byte es el número de un tile. Es del tamaño exacto del background, así que **los mismos bytes** se copian a la pantalla (`map_load`) y se usan para saber qué es sólido.

```
columna:  0  1  2  3 ... 31
fila 0:  [T][T][T][T] ... [T]      ← bytes 0 a 31
fila 1:  [T][.][.][.] ... [T]      ← bytes 32 a 63
...
fila 31: [T][T][T][T] ... [T]      ← bytes 992 a 1023
```

El mapa está dibujado con caracteres en `assets/levels/maps.asm` (una letra por tile): el ensamblador los convierte en números con `CHARMAP` .

De un punto del mundo al tile que lo cubre

Un tile mide **8 × 8 píxeles**. Para un punto `(Y, X)` del mundo:

```
fila    = Y / 8
columna = X / 8
dirección = mapa + fila × 32 + columna
```

Es una cuenta de potencias de 2, así que se hace con desplazamientos de bits (en inglés, *shifts*), no con multiplicaciones:

- `Y / 8` seguido de `× 32` es `Y` con sus 3 bits bajos a cero, multiplicado por 4: `(Y & %11111000) × 4` . Lo primero se hace con `and` ; el `× 4` son dos `add hl, hl` (cada uno duplica el valor de 16 bits en `HL`).
- `X / 8` son 3 desplazamientos a la derecha (`srl a` tres veces).
- Se suma la columna a la fila: como `fila × 32` tiene sus 5 bits bajos a 0 y la columna es menor que 32, **nunca hay acarreo**: solo cambia `L` .
- Por último se suma la dirección del mapa (`map_current`), que son 16 bits: `add` en el byte bajo y `adc` en el alto (`adc` suma también el acarreo).

Mirado en bits, el offset del tile (`fila × 32 + columna`) es la unión de los 5 bits altos de `Y` (la fila) y los 5 bits altos de `X` (la columna). Por eso nunca hay acarreo entre ellos:

```
   Y = y7 y6 y5 y4 y3 y2 y1 y0       X = x7 x6 x5 x4 x3 x2 x1 x0
       └──── fila ────┘                   └─ columna ──┘

   offset         = y7 y6 y5 y4 y3 x7 x6 x5 x4 x3    (10 bits)
                    └ fila × 32 ─┘ └─ columna ──┘
```

ⓘ Un ejemplo para comprobar tu cuenta

Punto `Y = 216`, `X = 32`. Fila = 27, columna = 4, offset = `27 × 32 + 4 = 868`. Si el mapa empezara en `$0188`, la dirección sería `$0188 + $0364 = $04EC`.

```
columna:  0  1  2  3  4 ... 31
fila 27: [T][.][.][.][●] ... [T]      ← bytes 864 a 895; ● es el 868
```

La rutina que hace esto es `map_tile_address` (`D` = Y, `E` = X, devuelve `HL`).

¿Es sólido?

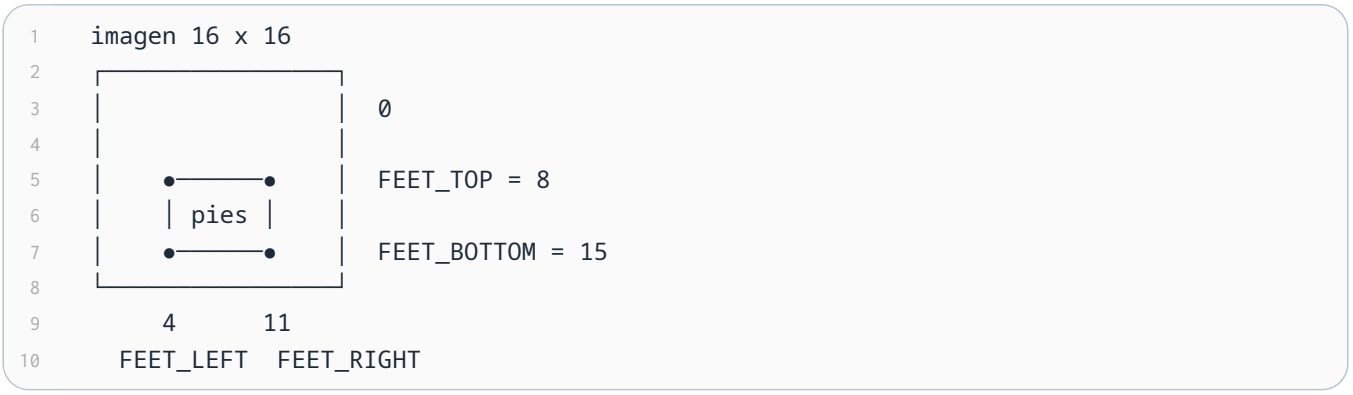
Los tiles están **ordenados**: del número `TILE_FIRST_SOLID` en adelante, todos son sólidos (árbol, roca, agua, muro, ladrillo). Así, una sola comparación decide:

```
map_is_solid_at(D = Y, E = X):
    HL = map_tile_address(...)
    A  = el tile que hay en HL
    si A < TILE_FIRST_SOLID:  devolver Z   (libre)
    devolver NZ                              (sólido)
```

Cuidado con el contrato: `NZ` **significa sólido** y `Z` **significa libre**.

Cómo se usa en la física: la caja de los pies

Una entidad mide 16 × 16, pero no choca con todo su cuerpo: choca con sus **pies**, una caja de **8 × 8** píxeles. Si los pies caben en un tile, comprobar sus **4 esquinas** basta: cualquier tile que toquen tiene una esquina dentro.



`sys_physics_blocked` mira esas 4 esquinas (con `map_is_solid_at`) y devuelve `NZ` si alguna está en un tile sólido. Y `sys_physics_one` mueve **cada eje por separado**: primero X (y si los pies quedan en un tile sólido, deshace ese movimiento) y luego Y. Así, si caminas en diagonal contra una pared, **te deslizas** por ella.

Cómo plantear el ejercicio 3

En el ejercicio 3 escribes `map_tile_address` (la suma final de `map_current`, con `adc`, ya está dada) y después `map_is_solid_at`, que sigue el pseudocódigo de «¿Es sólido?». Antes de escribir una instrucción, haz la cuenta de arriba en papel con un punto concreto y decide qué registro guarda cada resultado:

1. `Y & %11111000` es `fila × 8`. Hay que pasarlo a `HL` de 16 bits: `H` = 0 y `L` = ese valor.
2. Dos `add hl, hl` y `HL` = `fila × 32`.

3. `X / 8` es la columna. Se calcula en `A`, con tres desplazamientos.
4. Se suma a `L`. `HL` queda con el offset y el código dado hace el resto.

Tropiezos habituales: duplicar `HL` antes de poner `H` a 0, desplazar `X` más o menos de 3 veces, perder `fila × 8` al usar `A` para otra cosa y, en `map_is_solid_at`, devolver los flags que deja `cp`: con un árbol (`TILE_TREE` = `TILE_FIRST_SOLID`), `cp` deja `Z` a 1 aunque el tile sea sólido. Si te pasa algo, traza con `Y` = 216, `X` = 32 (el offset tiene que dar 868).

Bloque 4 · El HUD con la window

Ejercicio 4 · ~15 minutos de lectura

Una capa que no se mueve

El HUD (corazones, nivel y enemigos) tiene que verse siempre en el mismo sitio aunque el background haga scroll. Para eso existe la **window**: una capa que se dibuja *encima* del background y **no hace scroll**.

- Tiene su propio mapa de tiles, en `$9C00` (`WINDOW_MAP`). El background usa `$9800`.
- Su posición se controla con dos registros:
  - `rWY` (`$FF4A`): la línea de la pantalla donde empieza.
  - `rWX` (`$FF4B`): la columna donde empieza **más 7** (*WX = columna + 7*). Para empezar en el borde izquierdo, `WX = 7`.
- La window se dibuja desde `(WX - 7, WY)` hasta el final de la pantalla.

En CHELDA el HUD es **una fila de tiles (8 píxeles) en la parte de abajo**: `HUD_WINDOW_X = 7` y `HUD_WINDOW_Y = 144 - 8 = 136`.

Para ocultarla se pone `WY` por debajo de la pantalla (`HUD_HIDDEN_Y = 144`): `hud_hide`. Las pantallas de texto (menú, game over…) no tienen HUD.

> ⓘ **Los sprites van por encima de la window**
> Los sprites se dibujan sobre la window. Para que ninguna entidad tape el HUD, el pre-render no dibuja las que lo alcanzarían (lo verás en el bloque 7).

Dibujar los corazones

El HUD son 3 celdas de corazones, la palabra `NIVEL` con su número y los enemigos que quedan. La parte de los corazones recibe en `C` cuántos corazones tiene el héroe y escribe `HEARTS_MAX` celdas desde `HUD_HEARTS_CELL`:

- las primeras `C` celdas llevan `TILE_HEART_FULL` (lleno),
- el resto llevan `TILE_HEART_EMPTY` (vacío).

En pseudocódigo:

```
hud_draw_hearts(C = corazones):
    B  = HEARTS_MAX             ;; celdas por dibujar
    HL = HUD_HEARTS_CELL        ;; primera celda
    repetir:
        si C == 0:  tile = vacío
        si no:      tile = lleno, y C = C - 1
        escribir tile en [HL], y HL avanza una celda
        B = B - 1
    mientras B != 0
```

Piensa en `C` como "corazones llenos que aún me quedan por pintar". Un detalle de la CPU: `ld a, <valor>` **no** cambia los flags, así que puedes comprobar `C == 0` (con `ld a, c` / `or a`) y cargar el tile *después*, sin perder el resultado de esa comprobación.

Cuándo se redibuja

El HUD solo se redibuja si algo ha cambiado (`hud_dirty`). El juego avisa con `hud_refresh` , y `hud_update` (el último paso del frame, aún en VBLANK) vuelve a dibujarlo.

Comprueba que lo entiendes ›

1. ¿Qué valor de `rWX` pone la window en el borde izquierdo de la pantalla?
2. Para `C = 1` y `HEARTS_MAX = 3` , ¿qué tiles se escriben, en orden?

Respuestas al final de esta guía.

Bloque 5 · Los niveles: formato y carga

Ejercicio 5 · ~20 minutos de lectura

Un nivel son datos

Un nivel no tiene código: son **datos en la ROM** (`assets/levels/levels.asm`). Todos tienen el mismo formato:

Offset	Campo	Qué es
0	`LVL_MAP` (2 bytes)	Dirección de su mapa (1.024 tiles)
2	`LVL_SONG` (2 bytes)	Dirección de su canción
4	`LVL_PLAYER_TX` , `LVL_PLAYER_TY`	Dónde empieza el héroe: columna y fila, **en tiles**
6	`LVL_EXIT_TX` , `LVL_EXIT_TY`	Dónde aparece el portal
8	`LVL_ENEMIES`	Cuántos enemigos hay (N)
9	`LVL_ENEMY_LIST`	N pares de bytes: columna y fila de cada enemigo

Un nivel con 3 enemigos, por ejemplo:

```
DW Level1Map            ;; su mapa
DW SongLevel1           ;; su canción
DB 4, 27                ;; el héroe empieza en la columna 4, fila 27
DB 29, 2                ;; el portal aparece en la columna 29, fila 2
DB 3                    ;; hay 3 enemigos:
DB 18, 10,  27, 16,  19, 20
```

Las posiciones están en **tiles**. El mundo está en **píxeles**: hay que multiplicar por 8.

Los niveles están en una tabla (`LevelTable`) de direcciones de 16 bits, una por nivel. La rutina `level_get` devuelve en `HL` la dirección de los datos del nivel actual (`level_current`): `LevelTable + level_current × 2` , y luego lee ahí la dirección guardada (16 bits, byte bajo primero).

> **Para añadir un nivel**
> Dibujas su mapa, escribes sus datos, lo añades a `LevelTable` y subes `NUM_LEVELS` . No tocas ninguna rutina.

Cargar un nivel

`game_level_load` carga el nivel actual con la pantalla apagada:

1. Dibuja su mapa (`map_load`).
2. Crea al héroe en su tile de inicio.
3. Crea a los enemigos (`game_enemies_spawn`).

Para crear una entidad en un tile se usa `game_spawn_at` :

- 📥 `HL` = plantilla de la entidad, `C` = columna, `B` = fila (en tiles).
- Crea la entidad desde la plantilla (`man_entity_create`) y escribe su posición en píxeles: `Y = fila × 8` , `X = columna × 8` (tres veces `add a`).
- **No modifica** `DE` . Eso te permite usar `DE` como puntero a la lista de enemigos mientras creas.

Crear a los enemigos: `game_enemies_spawn`

Recibe en `HL` la dirección del campo `LVL_ENEMIES` del nivel:

```
game_enemies_spawn(HL = &level.ENEMIES):
    N = el byte de HL (y HL avanza)
    enemies_left = N
    DE = HL                       ;; DE recorre la lista
    si N == 0:  volver
    repetir N veces:
        C = [DE] (columna), DE avanza
        B = [DE] (fila),    DE avanza
        HL = EnemyTemplate
        game_spawn_at(HL, C, B)
```

Dos cuidados:

- `game_spawn_at` modifica `A` , `BC` y `HL` , pero **no** `DE` . El contador de enemigos que te quedan por crear no puede estar en `B` ni en `C` : guárdalo en la pila (`push af` / `pop af`) o en otro sitio seguro.
- `enemies_left` es un dato **de este módulo** (las reglas del juego): lo escribes tú, y el HUD lo lee con `game_get_enemies` .

Pasar de nivel

Al tocar el portal, el juego incrementa `level_current` y cambia **a la misma escena** (`scene_game`): su `init` carga el nivel nuevo. Tras el último, cambia a la escena de victoria.

Bloque 6 · IA básica

Ejercicios 6 y 7 · ~20 minutos de lectura

La idea: decidir poco y de vez en cuando

Un enemigo no piensa en cada frame. Cada cierto tiempo **elige** qué hacer: o se queda quieto, o camina en una dirección al azar. Entre una elección y la siguiente, **sigue con lo que eligió**: su velocidad ya está guardada, y la física lo mueve.

La IA solo **decide**. Caminar o pararse lo hacen dos rutinas de la física, que escriben la velocidad y la dirección:

- `sys_physics_walk`: `DE` = entidad, `B` = dirección (`DIR_DOWN`, `DIR_UP`, `DIR_LEFT`, `DIR_RIGHT`). La entidad mira ahí y se mueve a esa velocidad.
- `sys_physics_stop`: `DE` = entidad. Se queda quieta (y sigue mirando hacia donde miraba).

El temporizador `THINK`

El campo `THINK` cuenta los frames que faltan para decidir. En cada frame:

1. Se le resta 1.
2. Si no ha llegado a 0, la IA **no hace nada más**.
3. Si ha llegado a 0, toca decidir y se pone un nuevo `THINK`: `AI_THINK_MIN` (24) más un número aleatorio de 5 bits (0-31). Entre 24 y 55 frames: de 0,4 a 0,9 segundos.

```
sys_ai_one(DE = entidad):
    THINK = THINK - 1
    si THINK != 0:  volver
    A = número aleatorio                ;; rng_next
    THINK = AI_THINK_MIN + (A AND AI_THINK_MASK)
    elegir qué hacer con el MISMO número aleatorio   ;; sys_ai_choose
```

Los números aleatorios

`rng_next` devuelve en `A` un número de 0 a 255 (un generador *xorshift* de 16 bits). **Modifica `A`, `B` y `HL`**. Es la causa de un error típico: si guardaste en `HL` la dirección de `THINK`, `rng_next` te la sobreescribe. Hay que guardarla (`push hl` / `pop hl`) y guardar también el número aleatorio, porque se usa dos veces: para el nuevo `THINK` y para elegir.

Elegir con un número aleatorio: `sys_ai_choose`

Con el mismo número aleatorio `A` se decide qué hacer:

- El **bit 7** de `A` dice si se queda quieto: a 1 (la mitad de las veces) se llama a `sys_physics_stop`.
- Si no, la dirección son los **bits 6-5** de `A`, un número de 0 a 3 que coincide con `DIR_DOWN`, `DIR_UP`, `DIR_LEFT`, `DIR_RIGHT`. Se llama a `sys_physics_walk` con `B` = esa dirección.

```
  A = b7 b6 b5 b4 b3 b2 b1 b0
      |  └─┴─ dirección (0-3)
      └ 1: quieto
```

Para llegar a esos bits con pocas instrucciones hay varios caminos. Uno: `swap a` (intercambia sus dos mitades de 4 bits, los *nibbles*) deja los bits 7-5 en las posiciones 3-1; `rrca` los baja una posición más y deja el "quieto" en el bit 2 y la dirección en los bits 1-0. Entonces `bit 2, a` pregunta por "quieto" y `and DIR_MASK` (`%00000011`) se queda con la dirección. Otro camino distinto vale igual si da el mismo resultado.

Los dos finales (`sys_physics_stop` y `sys_physics_walk`) terminan con su propio `ret`, así que puedes **saltar** a ellos con `jp` en vez de llamarlos: es una optimización que ya conoces de ***CHAsteroids*** (*tail call*).

Comprueba que lo entiendes ›

1. Con `A = %10110100`, ¿qué hace la entidad? ¿Y con `A = %01000111`?
2. ¿Por qué se usa el mismo número aleatorio para el nuevo `THINK` y para elegir?

Respuestas al final de esta guía.

Bloque 7 · De coordenadas del mundo a coordenadas de la OAM

Ejercicios 8 y 9 · ~20 minutos de lectura

Tres sistemas de coordenadas

En el juego conviven **tres** maneras de decir "dónde está" una entidad:

Sistema	Origen	Quién lo usa
Mundo	Esquina del mapa	`E_Y`, `E_X` de la entidad, la física, las colisiones
Pantalla	Esquina de lo que se ve	Lo que ve el jugador
OAM	Pantalla con un offset: **+16 en Y, +8 en X**	Los sprites de la OAM

Un ejemplo con una entidad (miramos su esquina de arriba a la izquierda) y la cámara en `(X = 50, Y = 20)`. El dibujo es un esquema, no está a escala:

```
(Y=0, X=0) Origen del MUNDO (background)
●──────────────────────────────────────────────────
│
│       Origen de la OAM (margen 16,8 respecto a PANTALLA)
│       ○-----------------------------------------
│       ¦  ◆─ origen de la PANTALLA (=cámara)─────┐
│       ¦  │ (Y=20,X=50) en el mundo              │
│       ¦  │                                      │
│       ¦  │                                      │
│       ¦  │                  ▉▉  entidad         │
│       ¦  │                      en el mundo:    Y=60, X=100
│       ¦  │                      en la pantalla: Y=40, X=50
│       ¦  │                      en la OAM:      Y=56, X=58
│       ¦  │                                      │
│       ¦  └──────────────────────────────────────┘
```

Cada sistema mide desde **su** origen, así que el mismo punto tiene tres posiciones distintas:

Sistema	Posición de la entidad	Cuenta
Mundo	`Y = 60`, `X = 100`	Es lo que guarda la entidad (`E_Y`, `E_X`)
Pantalla	`Y = 40`, `X = 50`	`60 - 20`, `100 - 50`: se resta la cámara
OAM	`Y = 56`, `X = 58`	`40 + 16`, `50 + 8`: la OAM empieza 16 más arriba y 8 más a la izquierda que la pantalla

La regla importante: **las entidades guardan su posición en coordenadas del mundo, y solo se cambia a coordenadas de pantalla al pintar**. Así la física y las colisiones no saben nada de la cámara.

El paso del mundo a la pantalla es restar la cámara:

```
pantalla = mundo - cámara
```

Y la OAM suma un offset a esa posición para que un sprite pueda estar **parcialmente fuera** de la pantalla (con `Y = 0` el sprite está justo escondido arriba; con `Y = 16`, justo en el borde):

```
OAM Y = mundo Y − cámara Y + OAM_Y_OFFSET     (OAM_Y_OFFSET = 16)
OAM X = mundo X − cámara X + OAM_X_OFFSET     (OAM_X_OFFSET = 8)
```

Todo en 8 bits, sin comprobar si se ve

Las tres cuentas se hacen en 8 bits. Si la entidad queda **fuera** de la pantalla, el resultado "da la vuelta" y sale un valor que la consola no dibuja: por ejemplo, una `Y` de OAM mayor o igual que 160, o una `X` de OAM mayor o igual que 168. **No hace falta comprobar si el sprite se ve**: la propia aritmética lo esconde.

Un ejemplo con cámara `(X = 50, Y = 20)`:

Entidad (Y, X)	Cuenta en Y	OAM Y	Cuenta en X	OAM X
(60, 100)	60 − 20 + 16	56	100 − 50 + 8	58
(2, 3)	2 − 20 + 16 = −2 → 254	254	3 − 50 + 8 = −39 → 217	217

La segunda entidad está por encima y a la izquierda de la cámara: 254 y 217 quedan fuera de lo que la consola dibuja.

La rutina `world_to_oam` hace justo estas dos cuentas: 📥 `D` = Y del mundo, `E` = X del mundo; ⬅ `B` = Y en la OAM, `C` = X en la OAM. Pide la cámara con `sys_camera_get`, que devuelve la cámara en `B` y `C` y por eso **destruye** `B` y `C` de la entrada: de ahí que la entrada vaya en `D` y `E`.

Una entidad = 2 sprites

Los sprites son de **8 × 16** píxeles. Un sprite de 8 × 16 no muestra un tile suelto: muestra **dos tiles consecutivos**, uno encima del otro. En la OAM solo se escribe el número del primero (el de arriba) y la consola toma también el siguiente (el de abajo): si el sprite lleva el tile `N`, dibuja `N` arriba y `N + 1` debajo.

Eso tiene dos consecuencias:

- Los dos tiles de cada pareja tienen que estar **seguidos** en el bloque de tiles.
- El número del primero tiene que ser **par**. La consola ignora el bit 0 en este modo, así que si escribes un número impar, por ejemplo 81, dibuja la pareja 80 y 81. Por eso las sheets empiezan en un número par y cada imagen ocupa 4 tiles.

Una entidad mide 16 × 16: se dibuja con **dos sprites, uno al lado del otro**, y su imagen son **4 tiles seguidos**, ordenados por columnas (primero la izquierda, de arriba abajo, y luego la derecha):

```
 OAM X          OAM X + 8        ● = (OAM X, OAM Y)
●──────────┬──────────┐
│ TILE     │ TILE+2   │
├──────────┼──────────┤
│ TILE+1   │ TILE+3   │
└──────────┴──────────┘
 sprite 0   sprite 1
```

- El **sprite 0** (izquierda) muestra `TILE` y `TILE + 1`. En la OAM se escribe `TILE`.
- El **sprite 1** (derecha) muestra `TILE + 2` y `TILE + 3`. En la OAM se escribe `TILE + 2`.

Cada sprite son 4 bytes en la Shadow OAM: `Y`, `X`, `TILE`, `ATTR`.

El volteo (flip)

Las entidades que miran a la izquierda no tienen sus propias imágenes: se usa la imagen de mirar a la derecha **volteada** con el bit `ATTR_XFLIP` de `ATTR` (bit 5). Pero ojo: voltear los sprites **no intercambia sus posiciones**. Si volteas cada mitad pero no las cambias de lado, la imagen queda rota.

```
sin voltear: [sprite 0: mitad izquierda][sprite 1: mitad derecha]
volteada:    [sprite 0 volteado va a la DERECHA][sprite 1 volteado va a la IZQUIERDA]
```

Los dos sprites tienen el mismo `ATTR` (con el flip). Lo que cambia es la `X` que se escribe en cada uno: con flip, el sprite 0 lleva `OAM X + 8` y el sprite 1 lleva `OAM X`.

No se dibuja lo que invade el HUD

Los sprites van por encima de la window. Una entidad cuyo borde de abajo alcance el HUD taparía los corazones. Así que `sys_prerender_one` **no la dibuja**:

- El borde de abajo del sprite está en la `Y` de la OAM (porque mide 16 y el offset de la OAM en Y es 16).
- Si esa `Y` es mayor que `HUD_WINDOW_Y` (136), no se dibuja. Eso incluye, gracias a la vuelta de 8 bits, lo que está por debajo de la pantalla.

El cursor de la Shadow OAM

Cada entidad escribe sus sprites en la Shadow OAM **a continuación** de la anterior. El sitio lo marca `shadow_cursor`, un dato de **1 byte** (el byte bajo de la dirección: la Shadow OAM está alineada a 256, igual que `entity_array`, así que el byte alto siempre es `HIGH(shadow_oam)`):

```
shadow_oam: [entidad 0: sprite 0 | sprite 1][entidad 1: ...][...
                                            ▲
                                       shadow_cursor
```

Al empezar el frame, `sys_prerender_update` vacía la Shadow OAM y pone `shadow_cursor` al principio; cada entidad dibujada lo avanza 8 bytes (2 sprites × 4).

La escritura de un sprite sale cómoda con `ld [hl+], a`: escribe y avanza `HL`.

Resumen de `sys_prerender_one`

```
sys_prerender_one(DE = entidad):
    D, E = Y y X del mundo de la entidad
    B, C = world_to_oam(D, E)
    si B > HUD_WINDOW_Y:  volver                    ;; no se dibuja
    D, E = TILE y ATTR de la entidad
    HL = shadow_oam + shadow_cursor
    sprite 0: Y = B, X = C (o C + 8 si hay flip), TILE = D,     ATTR = E
    sprite 1: Y = B, X = C + 8 (o C si hay flip), TILE = D + 2, ATTR = E
    shadow_cursor = el nuevo L
```

Comprueba que lo entiendes ›

1. Cámara `(X = 96, Y = 112)`, entidad en `(Y = 200, X = 150)`. ¿Cuál es su posición en la OAM?

2. ¿Por qué la entidad guarda su posición en el mundo y no en la pantalla?

Respuestas al final de esta guía.

Bloque 8 · Animación

Ejercicio 10 · ~20 minutos de lectura

Una sheet de imágenes

Para que el héroe parezca caminar no basta una imagen: hay varias, y en cada frame se muestra una u otra según hacia dónde mira y en qué paso va. Una **sheet** es el conjunto de las imágenes de una entidad, dibujadas una detrás de otra en el bloque de tiles. Cada entidad guarda dónde empieza la suya en su campo `SHEET` (la del héroe empieza en el tile 80).

Una sheet tiene **6 imágenes**: 3 direcciones (abajo, arriba y de lado) con 2 pasos de caminar cada una. Una imagen son 4 tiles (`TILES_PER_IMAGE = 4` : 2 sprites de 2 tiles, como en el bloque 7), así que una **fila** de la sheet son 8 tiles (`SHEET_ROW_TILES = 8`). Esta es la del héroe, con los tiles de cada imagen:

```
sheet del héroe (SHEET = 80)

           paso = 0          paso = 4
abajo      imagen 0: 80-83   imagen 1: 84-87     fila = 0
arriba     imagen 2: 88-91   imagen 3: 92-95     fila = 8
de lado    imagen 4: 96-99   imagen 5: 100-103   fila = 16
```

La fila "de lado" mira a la **derecha**. No hay imágenes para la izquierda: se usa la misma fila **volteada** con `ATTR_XFLIP` , como vimos en el bloque 7.

Qué imagen toca

Elegir la imagen son dos preguntas: ¿en qué **fila** está? (depende de hacia dónde mira, `DIR`) y ¿qué **paso** toca? (depende de cuánto lleva caminando, `TICK`). Con las dos respuestas:

```
TILE = SHEET + fila + paso
```

Por ejemplo, el héroe (`SHEET` = 80) mirando arriba, en su segundo paso: `80 + 8 + 4 = 92` , la imagen 3.

La fila: la tabla `DirImage`

Hay cuatro direcciones pero solo tres filas, y la izquierda necesita además el volteo. Por eso `sys_animation_one` usa una **tabla**, `DirImage` , con una entrada por dirección. Cada entrada son 2 bytes: el offset de su fila (dónde empieza dentro de la sheet) y el flip.

`DIR`	Fila (tiles)	Flip
`DIR_DOWN` (0)	0	no
`DIR_UP` (1)	8	no
`DIR_LEFT` (2)	16	`ATTR_XFLIP`
`DIR_RIGHT` (3)	16	no

La entrada de una dirección está en `DirImage + DIR × 2` : se multiplica por 2 porque cada entrada ocupa 2 bytes (un `add a`). Para leerla se suma ese índice a la dirección de la tabla, que son 16 bits (`ld hl, DirImage` y `add`

`hl, bc`).

El paso: `TICK`

`TICK` cuenta los frames que la entidad lleva caminando. La imagen cambia cada 8 frames, y el **bit 3** de `TICK` cambia justo cada 8 frames: vale 0 durante 8 frames, 1 durante otros 8, y así. Se aísla con `ANIM_STEP_MASK = %00001000`.

El problema es que ese bit, ya aislado, vale 0 u **8**, y el segundo paso empieza **4** tiles después (el tamaño de una imagen). Hay que dividir entre 2, con un `rrca`:

`TICK`	`TICK AND ANIM_STEP_MASK`	Dividido entre 2 = paso	Imagen de la fila
0-7	0	0	la primera
8-15	8	4	la segunda
16-23	0	0	la primera
24-31	8	4	la segunda

Si la entidad se para, `TICK` vuelve a 0 y enseña la primera imagen de su fila.

Un ejemplo: el héroe andando a la izquierda

El héroe (`SHEET` = 80) mira a la izquierda (`DIR_LEFT` = 2) y lleva `TICK` = 9 frames caminando:

1. **Fila y flip.** Entrada `2 × 2 = 4` de `DirImage`: fila = 16 y flip = `ATTR_XFLIP`.
2. **Paso.** `9 AND %00001000 = 8`, y `8 / 2 = 4`.
3. **Tile.** `TILE = 80 + 16 + 4 = 100`: la imagen 5 (la segunda de la fila de lado).
4. **Atributos.** `ATTR` lleva `ATTR_XFLIP` activo: la imagen 5 se dibuja volteada y el héroe mira a la izquierda.

La rutina completa

```
sys_animation_one(DE = entidad con ANIMATION):
    si la entidad tiene PHYSICS:
        si VY y VX son los dos 0 (está quieta):
            TICK = 0                 ;; vuelve a su primera imagen
            ir a "la imagen"
    TICK = TICK + 1                  ;; un frame más caminando (o animándose siempre)
  la imagen:
    paso   = (TICK AND ANIM_STEP_MASK) / 2          ;; 0 o 4
    fila   = DirImage[DIR × 2]                      ;; offset de la fila
    flip   = DirImage[DIR × 2 + 1]
    TILE   = SHEET + fila + paso
    ATTR   = (ATTR con el bit XFLIP a 0) OR flip
```

Fíjate en dos detalles:

- Una entidad **sin** `PHYSICS` (el portal) anima **siempre**: nunca está "caminando", pero su imagen debe cambiar.
- `ATTR` puede traer otros bits (el parpadeo del héroe herido usa el de la paleta). Por eso el flip se escribe borrando **solo el bit** `ATTR_XFLIP` y poniéndolo de nuevo según la dirección, sin tocar los demás.

Leer código con errores (otra vez)

El ejercicio 10 es de corregir. Aplica lo del bloque 1: escribe antes lo que cada bloque debería hacer y compáralo. Sitios donde suelen esconderse los errores en esta rutina: la condición que decide si se anima (`jr z` / `jr nz`), el cálculo del paso (qué bit, cuánto se desplaza), el índice de la tabla (`DIR × 2`), el nombre de las etiquetas y qué bit de `ATTR` se borra.

Último bloque: música y efectos

CHELDA tiene música y efectos. No hay ejercicios de sonido en este Challenge, pero es útil ver cómo está hecho, porque lo usarás en tu proyecto.

- La **música** la compone una persona en un *tracker* (**hUGETracker**) y la toca un *driver* (**hUGEDriver**, en `engine/lib/hUGEDriver/`). Las canciones son datos: `assets/snd/song_*.asm`.
- `sound_play_song` empieza una canción (`HL` = la canción). `sound_update` toca **un tick**: se llama **una vez por frame**, siempre en el mismo sitio (al final del bucle, tras el render), para que el tempo sea constante.
- Cada escena empieza su canción en su `init`; cada nivel tiene la suya, en sus datos (`LVL_SONG`).
- La música usa los canales 1 y 2 (pulso) y 3 (onda). El **canal 4 (ruido) queda libre para los efectos**. Un efecto son 4 bytes que se copian a los registros del canal 4 (`rAUD4LEN`, `rAUD4ENV`, `rAUD4POLY`, `rAUD4GO`): `assets/snd/snd_effects.asm` explica cada byte y hay tres efectos (espada, enemigo derrotado, héroe herido).
- Si quieres música propia: **hUGETracker** exporta un `.asm` en formato RGBDS. Colócalo en `assets/snd/`, dale un nombre de sección propio (`SECTION "Song: ..."`) y llama a `sound_play_song` con su etiqueta. Y recuerda **citar** el trabajo de otras personas en tu `README.md`.

Y después

Cuando termines los ejercicios, intenta explicar **por qué** funciona cada uno: qué hace cada bloque y qué pasaría sin él. Eso te permitirá adaptarlo en clase.

> **Reflexión**
> Este Challenge mueve datos entre tres sistemas de coordenadas, y casi todas las rutinas se apoyan en una idea: **guardar las cosas en su sistema natural y convertir solo al final**. ¿Dónde más lo has visto?

Respuestas a «Comprueba que lo entiendes»

Bloque 1 · DMA: copiar sprites a la OAM

1. Porque la CPU solo puede leer HRAM durante el DMA; el `call` iría a la ROM.
2. 1.600 − 1.140 = unos 460 ciclos.

Bloque 2 · Scroll y cámara

1. `20 - 72` da acarreo → 0; `200 - 72 = 128` es mayor que 96 → 96.
2. Porque el resultado de la resta, tras "dar la vuelta", sería un número grande (p. ej. 214) y la cámara saltaría al otro extremo del mundo.

Bloque 4 · El HUD con la window

1. 7.
2. lleno, vacío, vacío.

Bloque 6 · IA básica

1. El bit 7 está a 1: se queda quieta. Con `%01000111`: el bit 7 es 0 y los bits 6-5 son `10` = 2 = `DIR_LEFT`: camina hacia la izquierda.
2. Porque `rng_next` es una rutina costosa y un solo número da bits de sobra para las dos cosas; los bits que usa el temporizador (los 5 bajos) no son los que usa la elección (los 3 altos).

Bloque 7 · De coordenadas del mundo a coordenadas de la OAM

1. Y: 200 − 112 + 16 = 104; X: 150 − 96 + 8 = 62.
2. Porque entonces la física y las colisiones no necesitan la cámara y no hay que corregir las posiciones de todas las entidades cada vez que la cámara se mueve.